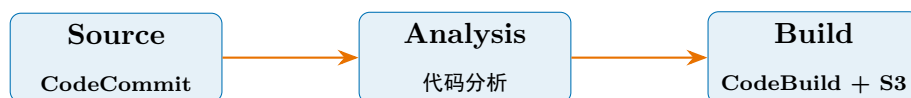


AWS Jam 挑战题总结

CodePipeline 集成代码检查（Lint）

题目背景

某公司已构建了一个用于生成应用程序代码的 AWS CodePipeline，流水线分为三个顺序阶段：



- **Source（来源）**：CodeCommit 仓库，存储源代码
- **Analysis（分析）**：CodeBuild 执行代码分析
- **Build（构建）**：CodeBuild 构建，产物存储至 S3

挑战任务

注意

在不增加构建时间的前提下，向现有 CodePipeline 中添加代码检查（Lint）规范强制执行机制。

挑战环境已预先创建的资源：

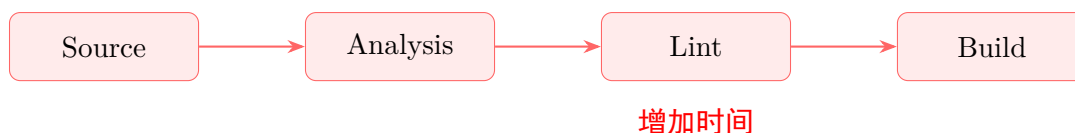
- CodeCommit 仓库（含源码和 buildspec 文件）
- S3 存储桶（存储 Pipeline 工件）
- CodePipeline 的 IAM 角色及内联策略

- CodePipeline 实例
- CodeBuild 项目（含预建的 LintProject）

核心思路

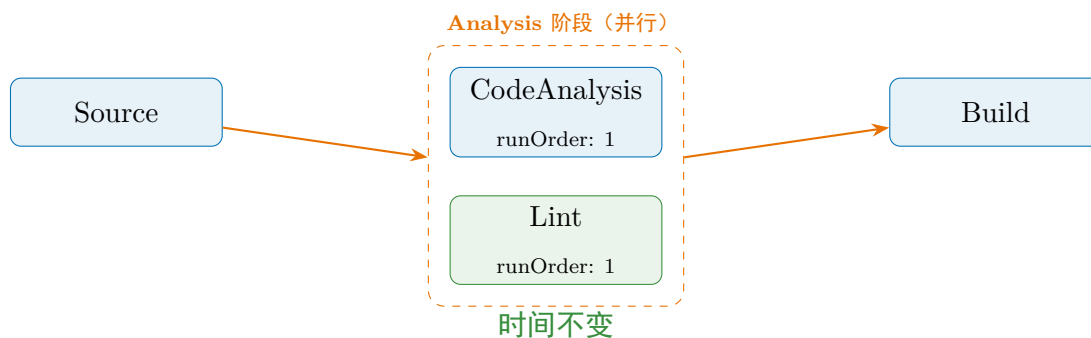
问题分析

若将 Lint 作为独立的新阶段串行添加，必然增加整体构建时间：



解决方案：同阶段并行执行

将 Lint 动作加入 **Analysis** 阶段，与现有分析动作并行运行（runOrder 设为相同值）：



关键结论

CodePipeline 同一 Stage 内，runOrder 相同的多个 Action 会并行执行。这意味着耗时不会像串行阶段那样直接相加，但整个 Stage 的完成时间仍取决于最慢的那个 Action。

操作步骤

第一步：进入 CodePipeline 控制台，编辑流水线

1. 打开 AWS Console → CodePipeline → 选择目标 Pipeline
2. 点击右上角 编辑
3. 找到 **Analysis** 阶段，点击 添加操作

第二步：配置 Lint 操作（填写编辑操作表单）

字段	填写值	说明
操作名称	Lint	自定义名称
操作提供程序	AWS CodeBuild	由 CodeBuild 执行
区域	美国（弗吉尼亚北部）	与 Pipeline 同区域
输入构件	SourceArtifact	使用源码作为输入
项目名称	LintProject-xxx	选择预建的 Lint 项目
构建类型	单次构建	触发一次构建实例

关键配置

确保 Lint Action 与 CodeAnalysis Action 的 **运行顺序（Run Order）** 均为 1，这是实现并行执行的核心。

第三步：保存并发布更改

1. 点击 **完成** 保存操作配置
2. 点击 **保存** 保存 Pipeline
3. 在“发布更改”对话框中，**勾选**“此更新无需资源更新”
4. 点击确认，Pipeline 自动触发执行

知识点详解

CodePipeline 核心概念

操作提供程序（Action Provider）

知识点

CodePipeline 本身只是**调度器**，不执行具体任务，需要指定提供程序来真正干活。

提供程序	用途
AWS CodeBuild	运行构建、测试、Lint 等脚本
AWS CodeDeploy	部署应用到服务器
AWS Lambda	执行自定义无服务器函数
Amazon S3	上传/下载工件文件

输入构件（Input Artifact）

每个 Stage 会产出一个”工件包裹”，下游 Action 通过输入构件声明使用哪个包裹作为原料：

```

1 Source 阶段  --产出-->  SourceArtifact（源代码压缩包）
2
3                               |
3                               v
4 Analysis/Lint 阶段  <--输入-- SourceArtifact（直接分析源码）
5
6                               |
6                               v
7 Build 阶段  <--输入-- AnalysisOutput（上阶段产出）
  
```

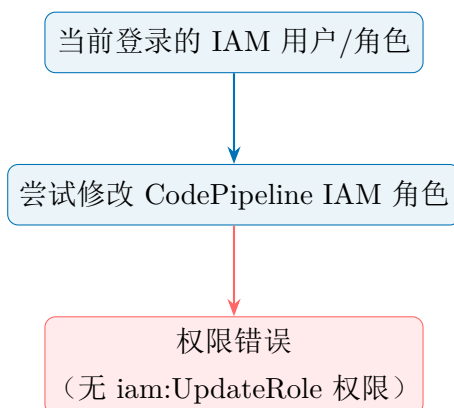
Listing 1: 构件流转示意

构建类型（Build Type）

类型	说明	适用场景
单次构建	启动 1 个 CodeBuild 实例	普通构建/Lint
批处理构建	同时启动多个实例	多平台/多版本并行测试

”此更新无需资源更新”选项

为什么不勾选会报权限错误？



为什么勾选后正常工作？

知识点

挑战环境已预先配置好所有 IAM 权限，无需 AWS 自动更新角色。勾选该选项等于告诉 AWS：“IAM 已经 OK，跳过角色更新步骤。”

```
1 # 不勾选（报错）
2 Pipeline 保存 -> AWS 尝试更新 IAM 角色 -> 无权限 -> 失败
3
4 # 勾选（成功）
5 Pipeline 保存 -> 跳过 IAM 更新 -> 直接保存 -> 成功
```

Listing 2: 两种情况对比

CodeBuild 与 Lint 的关系

知识点

AWS CodeBuild 没有内置 Lint 工具，但提供了标准化的运行环境。通过在 `buildspec.yml` 中安装和调用 Lint 工具来实现代码检查。

```
1 version: 0.2
2
3 phases:
4   install:
5     runtime-versions:
6       python: 3.11
7     commands:
8       - pip install pylint --quiet # 安装 Lint 工具
9
10  pre_build:
11    commands:
12      - echo "Running lint check..."
13      - pylint src/**/*.py # 执行 Lint
14
15  build:
16    commands:
17      - echo "Building application..."
18
19  artifacts:
20    files:
21      - '**/*'
```

Listing 3: buildspec.yml 中集成 Lint 示例

常用 Lint 工具对照：

语言	Lint 工具	安装命令
Python	pylint / flake8	<code>pip install pylint</code>
JavaScript	ESLint	<code>npm install -g eslint</code>
Ruby	RuboCop	<code>gem install rubocop</code>
Go	golint	<code>go install</code> <code>golang.org/x/lint</code>

成功验证标准

检查项	预期结果
Pipeline 结构	Analysis 阶段显示两个并行 Action
构建时间	与原来相比无明显增加
Lint 失败时	Pipeline 在 Analysis 阶段停止, 不进入 Build
挑战标记	CodePipeline 成功执行后自动标记挑战完成

总结

关键结论

核心知识点一句话概括：

利用 CodePipeline 同一 Stage 内 `runOrder` 相同的 Action 并行执行的特性，将 Lint 与现有 Analysis 同步运行，**避免串行增加时间**；若 Lint 不比现有分析更慢，通常不会明显拉长 Stage。

IAM 权限一句话概括：

挑战环境预配置了所有权限，勾选“此更新无需资源更新”跳过不必要的 IAM 角色更新，避免因缺少 `iam:UpdateRole` 权限而报错。